

## Lab4 Advanced: Keyboard & Audio

### Submission Due Dates:

|              |                  |
|--------------|------------------|
| Demo:        | 2025/10/21 17:20 |
| Source Code: | 2025/10/21 18:30 |
| Report:      | 2025/10/26 23:59 |

### Objective

This advanced lab will strengthen your FPGA design skills by integrating FSM design with external peripherals (keyboard and audio). Specifically, you will:

- Gain experience modeling finite state machines in Verilog.
- Learn to interface with a PS/2 keyboard on the FPGA demo board.
- Control the audio peripheral Pmod I2S2 to generate sound.

### Action Items

#### Part 1: lab4\_1 Fibonacci-Style Sequence Generator (20%)

##### A. Description

Implement a Fibonacci-style sequence generator on the **Basys 3 FPGA** board using keyboard input. The goal is to allow the user to input a series of numbers  $K[i]$  through a keyboard, and compute the sequence  $S[i]$  based on the following recurrence relation:

$$S[0] = 0$$

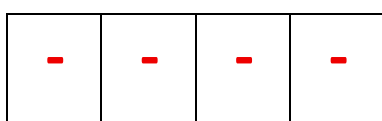
$$S[i] = (K[i] + S[i-1]) \% 10, \text{ for } i \geq 1$$

Each time the user inputs a number  $K[i]$ , the system calculates a new  $S[i]$  and displays the result on the **7-segment display**.

If the number of results exceeds the display digits, the display **scrolls left**, discarding the rightmost value.

##### Example Walkthrough

###### a. initial



**b.** press 5

$$S[1] = K[1] + S[0] = 5 + 0 = 5$$

|   |   |   |   |
|---|---|---|---|
| 5 | - | - | - |
|---|---|---|---|

**c.** press 6

$$S[2] = (K[2] + S[1]) \% 10 = (6 + 5) \% 10 = 1$$

|   |   |   |   |
|---|---|---|---|
| 1 | 5 | - | - |
|---|---|---|---|

**d.** press 7, press 8

$$S[3] = (K[3] + S[2]) \% 10 = (7 + 1) \% 10 = 8$$

$$S[4] = (K[4] + S[3]) \% 10 = (8 + 8) \% 10 = 6$$

|   |   |   |   |
|---|---|---|---|
| 6 | 8 | 1 | 5 |
|---|---|---|---|

**e.** press 9

$$S[5] = (K[5] + S[4]) \% 10 = (9 + 6) \% 10 = 5$$

|   |   |   |   |
|---|---|---|---|
| 5 | 6 | 8 | 1 |
|---|---|---|---|

**f.** press rst

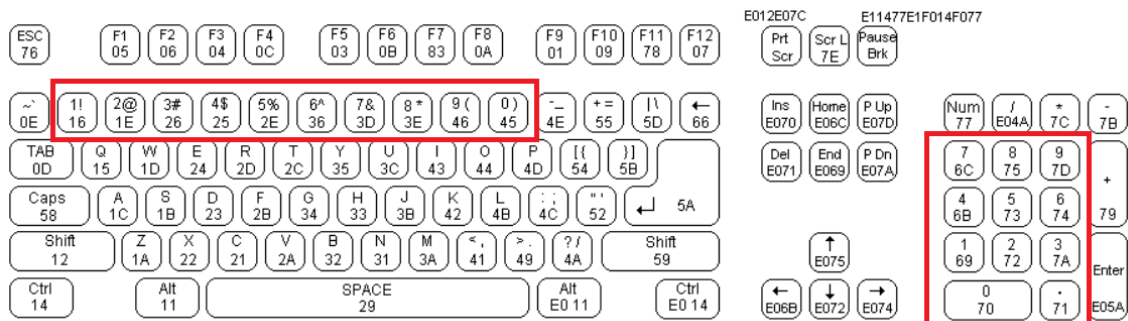
|   |   |   |   |
|---|---|---|---|
| - | - | - | - |
|---|---|---|---|

## B. I/O signal specification

| I/O          | Connected to | Definition                                |
|--------------|--------------|-------------------------------------------|
| clk          | W5           | Clock signal with the frequency of 100MHz |
| rst          | BTNU         | Asynchronous active-high reset            |
| digit[3:0]   |              | Signals to control the 7-segment digits   |
| display[6:0] |              | Signals to control 7-segment encoding     |
| PS2_DATA     |              | PS/2 keyboard data                        |
| PS2_CLK      |              | PS/2 keyboard clock                       |

## C. Rules & Hints

- After reset, the 7-segment display must show “----”.
- When a key is held down, no other key should activate until it is released.** For example, after you press and hold the key ‘2’, pressing the key ‘3’ will be invalid until you release the key ‘2’. If you still hold the key ‘3’ after releasing the key ‘2’, it is acceptable whether the key ‘3’ becomes active or not.
- A key press should only register once (no repeats while holding).** For example, after you press and hold the key ‘2’ when 7-segment display is “----”, the display must change to “2---” instead of “2222”.
- Do not modify any of the provided files. Otherwise, you will not receive full credit.**
- PS2\_CLK and PS2\_DATA are bidirectional (**inout**) signals. Don’t change their type.
- Both parts of the number keys will be tested in the demo.



- Use the following template for your design (you should determine the proper data type of each IO):

```
module lab4_1 (  
    input wire clk,  
    input wire rst,  
    inout wire PS2_DATA,  
    inout wire PS2_CLK,  
    output reg [3:0] digit,  
    output reg [6:0] display  
);  
  
/* Note that output ports can be either reg or wire.  
   * It depends on how you design your module. */  
// add you design here  
endmodule
```

**h. Demo video:**

[https://drive.google.com/file/d/1BrBA-b\\_tIKED-Z03NCxKMhMhGcQwWTJ8/view?usp=drive\\_link](https://drive.google.com/file/d/1BrBA-b_tIKED-Z03NCxKMhMhGcQwWTJ8/view?usp=drive_link)

## Part 2: lab4\_2 1A2B Number Guessing Game (40%)

### A. Description

Implement a classic "**1A2B**" **guessing game** using keyboard input and 7-segment display.

The game works as follows:

#### a. Secret Number Setup:

- Enter a unique 4-digit secret number using **number keys (0–9)**.
- Navigate (move) digit positions (left to right) with **the space bar**.
- After all 4 digits are entered, press the 'S' key to lock in the secret number and start guessing.

#### b. Guessing Phase:

- For each guess:
  - ✧ Enter a 4-digit number using **number keys (0–9)** and **the space bar** in the same way.
  - ✧ Press the '**S**' key to submit the guess. If the guess is correct, the system enters the Winning phase directly. Otherwise, it enters the Result Display phase.

### c. Result Display

- The design compares the guess with the secret number and displays the result in the format x**A**y**B**  
**A** : Correct digit in the correct position  
**B** : Correct digit but in the wrong position
- The incorrect result (e.g., 1A2B) will be shown for 3 seconds before returning to the Guessing Phase.

### d. Winning

- Display “**4A0B**” for correct guess. Then start a new game after 5 seconds.

## B. FSM specification

The FSM consists of five states: Initial State, Set State, Guess State, Result State, and Final State. A detailed description of each state is provided below:

### a. Initial state

- **7-segment**

The 7-segment shows “----”

- **LED**

All the LEDs are off.

- **Button**

Pressing **BTNR** to go to Set state

### b. Set state

Set the digits in this state. You can use the space key to switch between positions. Each digit must be in the range 0–9, and all digits must be unique. TAs will only test inputs that follow these rules, so you do not need to handle duplicates or out-of-range values.

- **Keyboard**

- ✧ Set the target number with key number 0-9.
- ✧ Use the space key to switch between which digit is currently selected. Initially, the leftmost digit is selected. When the space key is pressed, the selected position moves to the right. When the selection reaches the rightmost digit and you press the space key again, it will wrap around and return to the

leftmost digit.

✧ Pressing 'S' transitions to Guess state.

- **Setting and 7-segment Display**

The 7-segment display represents four digits from left to right. The leftmost digit corresponds to the 1st digit of the target number (e.g., for the number 5632, 5 is the 1st digit.), and the rightmost digit corresponds to the 4th digit of the target number (e.g., in the number 5632, 2 is the 4th digit.)

The 7-segment display initially shows "0000." When a keyboard key is pressed, the display will show the number currently being pressed at the selected digit. Each time the selection moves to a new digit, that digit will be reset to "0".

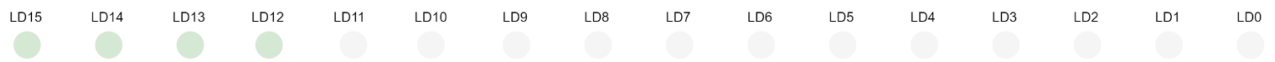
**Example:**

Initial display: "0000"

- Press 1 → "1000"
- Press space → "1000" (2nd digit selected, reset to 0)
- Press 2 → "1200"
- Press 6 → "1600" (the newly-entered 6 will replace the original 2)
- Press space → "1600" (3rd digit selected, reset to 0)
- Press 3 → "1630"
- Press space → "1630" (4th digit selected, reset to 0)
- Press 4 → "1634"
- Press space → "0634" (wraps to 1st digit, reset to 0)

- **LED Status**

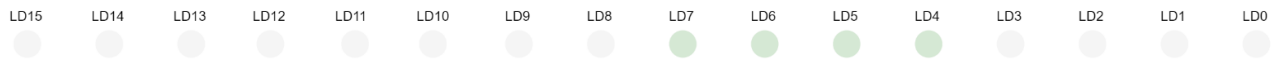
Set the 1<sup>st</sup> number: LED[15:12] on — others off



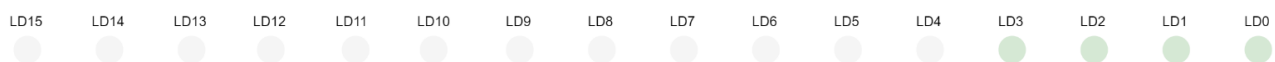
Set the 2<sup>nd</sup> number: LED[11:8] on — others off



Set the 3<sup>rd</sup> number: LED[7:4] on — others off



Set the 4<sup>th</sup> number: LED[3:0] on — others off



### c. Guess state

In this state, the player enters a 4-digit guess using the keyboard. The input method and 7-segment/LED display behavior are identical to those in the **Set** state.

After completing the input, pressing 'S' key will trigger the comparison between the guessed number and the secret number.

- If the guess matches the secret number, transition to the **Final** state.
- If the guess does not match the secret number, transition to the **Result** state.

### d. Result state

Display the result. Then return to Guess state after 3 seconds.

#### • 7-segment

Display xAyB result.

**Example:**



#### • LED

All the LEDs are off.

### e. Final state

Display the "**4A0B**" (since the guessed number is correct). Then return to **Initial** state

after 5 seconds.

- **7-segment**



- **LED**

The LEDs will be on for 1 second and off for 1 second. After **5** seconds (**on** → **off** → **on** → **off** → **on**), go back to the Initial state.

### C. I/O Signal Specification

| I/O          | Connected to | Definition                                                       |
|--------------|--------------|------------------------------------------------------------------|
| clk          | W5           | Clock signal with the frequency of 100MHz                        |
| rst          | BTNC         | Asynchronous active-high reset.<br>Reset to <b>Initial State</b> |
| set          | BTNR         | Enter <b>Set State</b> (from Initial)                            |
| LED[15:0]    |              | Indicates selected digit / flashes in Final State                |
| digit[3:0]   |              | 7-segment digit enable                                           |
| display[6:0] |              | 7-segment display control                                        |
| PS2_DATA     |              | Keyboard data                                                    |
| PS2_CLK      |              | Keyboard clock                                                   |

### D. Rules & Hints

- You must design at least one finite state machine (FSM). There must be at least five states. More states or multiple FSMs are acceptable. Explain your design in the report.
- Following the Rules & Hint (b)-(f) in Part 1.
- Use the following template for your design (you should determine the proper data type of each IO):

```
module lab4_2 (
    input wire clk,
    input wire rst,
    input wire set,
```



```

inout wire PS2_DATA,
inout wire PS2_CLK,
output reg [15:0] LED,
output reg [3:0] digit,
output reg [6:0] display
);

/* Note that output ports can be either reg or wire.
 * It depends on how you design your module. */
// add you design here
endmodule

```

**d. Demo video:**

[https://drive.google.com/file/d/1gwXFqVsazIZ7qC1YrxA10wGp6MBMCfQH/view?usp=drive\\_link](https://drive.google.com/file/d/1gwXFqVsazIZ7qC1YrxA10wGp6MBMCfQH/view?usp=drive_link)

## Part 3: lab4\_3 Guessing Note Game with Keyboard & Audio (40%)

### A. Description

Implement a note-guessing game controlled by FSM and keyboard, with sound generated by Pmod I2S2.

Notes are generated pseudo-randomly using a 9-bit LFSR.

### B. I/O signal specification

| I/O          | Connected to | Definition                                                       |
|--------------|--------------|------------------------------------------------------------------|
| clk          | W5           | Clock signal with the frequency of 100MHz                        |
| rst          | BTNC         | Asynchronous active-high reset.<br>Reset to <b>Initial State</b> |
| volume[2:0]  | LED[15:13]   | Indicate the volume                                              |
| LED[8:0]     |              | LFSR state                                                       |
| DIGIT[3:0]   |              | 7-segment digit enable                                           |
| DISPLAY[6:0] |              | 7-segment display control                                        |
| PS2_DATA     |              | Keyboard data                                                    |
| PS2_CLK      |              | Keyboard clock                                                   |
| Pmod JB 1~6  |              | Pmod I2S                                                         |

## C. Features

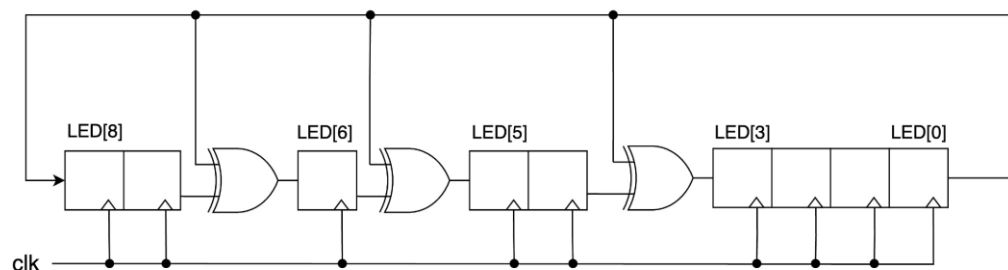
### ● Volume

- There are 3 levels (1-3) of volume, controlled by **Volume Up** (Key 'W') and **Volume Down** (Key 'S'). Initial volume level is 2.
- Hint: You can implement this feature based on the provided note\_gen.v.
- The three levels must be **heard and distinguishable**.
- Pressing Volume Up at level 3 or pressing Volume Down at level 1 takes no effect.
- LED 15~13 indicates the current volume level:

| Volume Level          | LED 15 | LED 14 | LED 13 |
|-----------------------|--------|--------|--------|
| Level 1               | ●      | ●      | ●      |
| Level 2               | ●      | ●      | ●      |
| Level 3 (the loudest) | ●      | ●      | ●      |

### ● LFSR

- Implement the 9bit Galois LFSR as follows:



- The state of the LFSR is displayed on LED[8:0].
- Pressing Key "N" shifts the LFSR one cycle to the right.
- Obtain the **note\_number** by calculating **LED[8:0] % 14**.
- Map the note\_number to notes C4-B5 as follows.

| Note_number | Note | Note_number | Note |
|-------------|------|-------------|------|
| 0           | C4   | 7           | C5   |
| 1           | D4   | 8           | D5   |
| 2           | E4   | 9           | E5   |
| 3           | F4   | 10          | F5   |
| 4           | G4   | 11          | G5   |

|   |    |    |    |
|---|----|----|----|
| 5 | A4 | 12 | A5 |
| 6 | B4 | 13 | B5 |

- Example of note mapping:

9'b101011001 (E5) -> 9'b111000100 (G4) -> 9'b011100010 (E4) ->  
9'b001110001 (D4)

## D. FSM Specification

The FSM must include three states: INITIAL, PLAY, and FINISH.

- In any time, **rst == 1** resets the FSM to INITIAL state.

### a. INITIAL state

- Set the LFSR state to 9'b101011001.
- Set the volume to Level 2.
- Set the 7-segment display to



- Transition to Guessing state on the next cycle.

### b. Guessing state

- Play the note that generated by the LFSR.
- A note composed of two parts: pitch and octave.
  - E.g., C4 -> pitch: C, octave: 4
- There are 7 pitches (C, D, E, F, G, A, B) in this lab.
- There are 2 octaves (4, 5) in this lab.
- Use keyboard to enter the pitch and octave.
- Enter the pitch first, and then the octave. Pressing number first take no effect. After inputting one pitch and one number, any input takes no effect.
- Pressing **"Backspace"** key deletes the previous input.
- When there is no previous input, **"Backspace"** does nothing.

- Pressing “Enter” key goes to Final state.
- Pressing “R” key goes to Initial state.
- For example,

- Initial 7-segment display



- Input octave ‘4’. (Nothing happens)



- Input pitch ‘C’.



- Input octave ‘4’.



- Pressing “Backspace” key



- Pressing “Backspace” key again



### c. FINAL state

- Display “GOOd” on the 7-segment display if the input matches the pseudo-random note generated by the LFSR. Otherwise, you should display “LOSE”.



- Pressing “Enter” key returns to Guessing state and reset the 7-segment display to:



- Pressing “R” key goes to Initial state.

### E. Demo Video

[https://drive.google.com/file/d/1xY7dlxBoXYs52iyT7CsbpICZWZ6xMJVj/view?usp=drive\\_link](https://drive.google.com/file/d/1xY7dlxBoXYs52iyT7CsbpICZWZ6xMJVj/view?usp=drive_link)

### F. Rules & Hints

- Use the template provided in template directory for your design.
- In the lab4\_3\_template directory, we also provide some example modules mentioned in the class. You may need them in your final project.
- Note-Frequency map

|           |     |     |     |     |     |     |     |
|-----------|-----|-----|-----|-----|-----|-----|-----|
| note      | C4  | D4  | E4  | F4  | G4  | A4  | B4  |
| frequency | 262 | 294 | 330 | 350 | 392 | 440 | 494 |
| note      | C5  | D5  | E5  | F5  | G5  | A5  | B5  |
| frequency | 523 | 587 | 659 | 699 | 784 | 880 | 988 |

## Questions and Discussion

Please answer the following questions in your report. Be clear and concise in explaining the design principles

- In lab4\_1, how did you prevent continuous detection of a positive signal when a key is pressed and held down?
- In lab4\_3, how did you control the volume? Please explain the principle.

## Report Guidelines

Your report should include but not limit to the following items.

- Lab Implementation (35%)
  - Block diagram of the design with explanation.
  - Description of each essential block.
  - State diagram(s) for FSM(s) with explanation, if applicable.
  - Code explanation with screenshot.

### 5. Questions & Discussions (50%)

Provide your answer to the Questions & Discussions in the lab assignment.

### 6. Problem Encountered (10%)

Describe the problems you encountered, solutions you developed, and the discussion. Explaining them with code segments or diagrams is recommended.

### 7. Suggestions (5%)

Optional: Feedback on the lab, course suggestions, or even a light-hearted joke.

## Attention

- Hand in **three Verilog files**: lab4\_1.v, lab4\_2.v, lab4\_3.v.
- Merge all modules into a single .v file for each part.
- Do **not** include modules such as KeyboardDecoder, onepulse, debounce, or speaker\_control in your submission.
- Do **not** submit compressed ZIP files, which will be considered an incorrect format.
- Report filename format: **lab4\_report\_StudentID.pdf** (e.g., lab4\_report\_123456789.pdf).
- You have the responsibility to check if the submission is successful.
- DO NOT submit ZIP files, which will be considered as incorrect format.
- Prepare bitstream files before demo to avoid delays.
- Be ready to answer TA questions during demo.
- Post questions on EECLASS forum if unclear.